CSE 299 · SECTION 3 · GROUP 4 · BASELINE UPDATE 1

How a Neural Network Learns

Built by hand in Python, step by step

Priyo Raihan (2311692042) · Sheanthea Akter (2413587642) · Marzia Anjum (2412117042)
Mustofa Nuruzzaman (2231929642) · Sayema Tasnim Aroni (2413241642)

THE PROJECT IN SIMPLE WORDS

What did we do?

WHAT

A small neural network, built by hand in Python

DATA

569 breast-cell samples, 3 measurements each

GOAL

Learn to tell malignant from benign

SO FAR

The mistake dropped from 0.16 to 0.10

> We wrote the learning steps ourselves, with no machine-learning library.

THE PROBLEM

Can a model learn from examples?

Give it 3 measurements of a cell. It should answer **1 = malignant** or **0 = benign**.

Learning is a loop of three moves:

1. Guess

The model predicts an answer.

2. Check

We measure how wrong it was.

3. Fix

We adjust the weights to be less wrong.

STEP 1 · GUESS

How the network makes a guess

1. 3 numbers go in.
2. Each of 4 hidden neurons mixes them using its **weights**.
3. **ReLU** sets any negative result to 0.
4. The output neuron mixes the 4 results into **one guess**.

```
h = ReLU(x · W01)
guess = h · W12
```

Example

Inputs: 2.30, 2.00, 2.11

Guess: **1.23**

True answer: **1**

Weight = how much an input matters. ReLU keeps the model flexible: without it, the layers collapse into one.

STEP 2 · CHECK

How wrong was the guess?

```
mistake = (guess - truth)²
```

Example

Guess 1.23, truth 1

$(1.23 - 1)^2 = 0.053$

Why square it?

- The mistake is never negative.
- Big mistakes count more.

STEP 3 · FIX

Change each weight a little

```
new weight = old weight - 0.01 × slope
```

0.01 is the learning rate (α)

Think of it like this

You are walking downhill in fog. Feel the slope, take a small step down, repeat. The hill is the mistake.

In our code

- Update after every sample
- 100 rounds (epochs) over the data
- Tiny steps: about 0.002 to 0.004 per weight

STEP 3B · BACKPROPAGATION

Who caused the mistake?

1. Start with the output mistake.
2. Send it **backward** to the hidden neurons.
3. Neurons that were off (ReLU = 0) get no blame.
4. Each weight is changed by: **input × blame**.

```
delta_2 = guess - truth
delta_1 = delta_2 · W12.T × (h > 0)
W12 -= 0.01 × h × delta_2
W01 -= 0.01 × x × delta_1
```

> This uses the **chain rule** from calculus: the mistake flows back through the same connections the guess flowed forward.

PUTTING IT TOGETHER

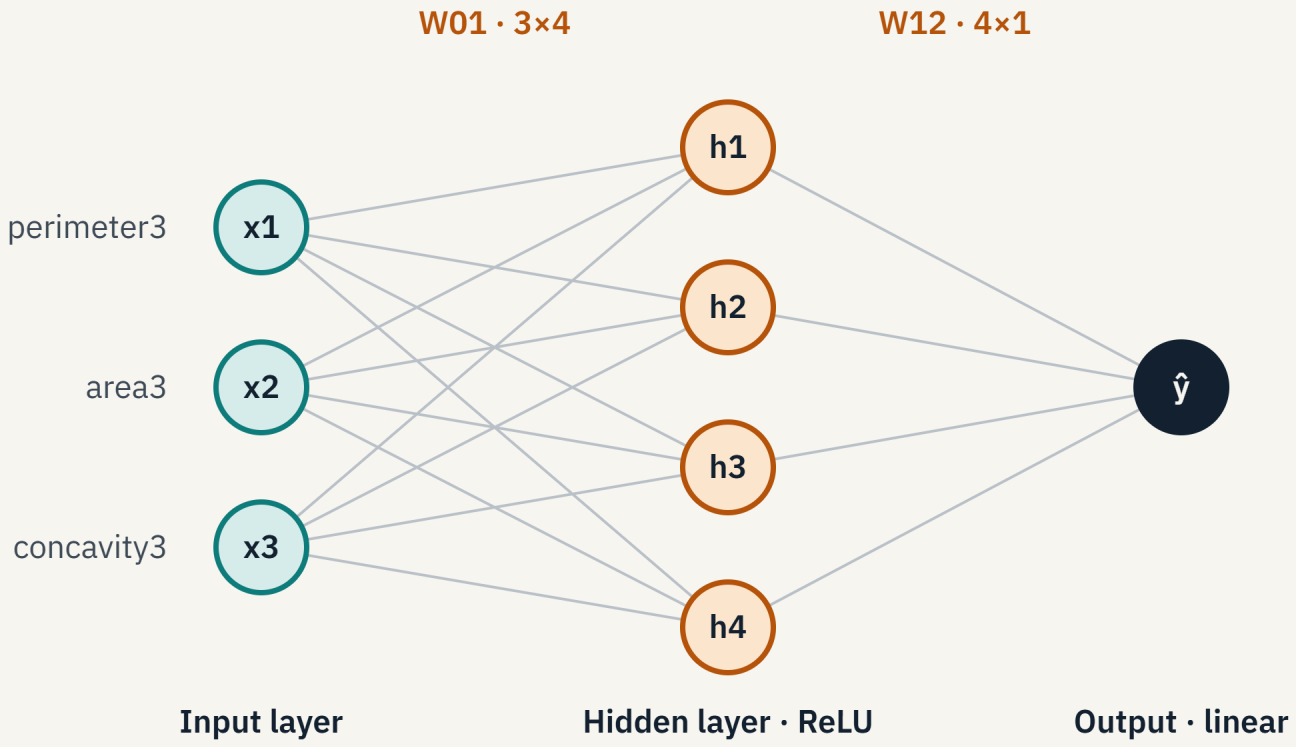
One example, start to end

Step	What happens for sample 0
1. Guess	The model says 1.23
2. Truth	The real answer is 1
3. Mistake	$(1.23 - 1)^2 = 0.053$
4. Blame	Output is too high by 0.23
5. Fix	Output weights go down by about 0.001 to 0.004

Calculated by us from the notebook's final weights. The notebook prints only the guess and the mistake.

SYSTEM ARCHITECTURE

The network we built: 3 → 4 → 1



Parameters

W01: 3 × 4 = 12 weights
W12: 4 × 1 = 4 weights
16 trainable weights, no biases

Fig. 2. Implemented architecture; every input feeds every hidden neuron.

HOW IT RUNS

From data to a trained model

1 · Load
Get the cell data

→

2 · Pick
Keep 3 features

→

3 · Scale
Same size for all

→

4 · Start
Random weights

→

5 · Train
100 rounds

Each training step repeats the same loop:

Guess → **Mistake** → **Blame** → **Fix**

THE CODE

The training loop, line by line

```
# for every epoch, for every sample:
hidden, pred = neural_network(input, weights)
error = ((pred - goal) ** 2).sum()
delta_2 = pred - goal
delta_1 = delta_2.dot(weights[1].T) * relu2deriv(hidden)
weights[1] = weights[1] - alpha * np.outer(hidden, delta_2)
weights[0] = weights[0] - alpha * np.outer(input, delta_1)
```

Guess · line 1

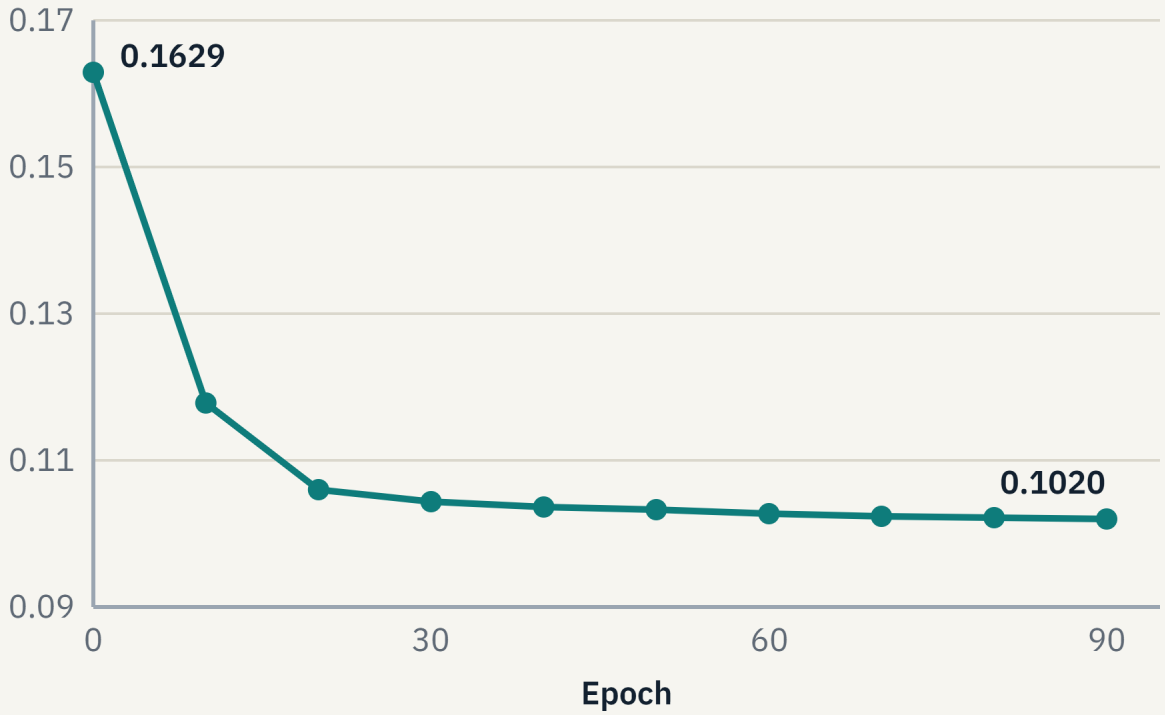
Mistake · line 2

Blame · lines 3 and 4

Fix · lines 5 and 6

RESULTS AND CURRENT PROGRESS

The mistake gets smaller



0.1629 → 0.1020

mean squared error, epoch 0 to epoch 90 (about 37% lower)

- Most of the drop is in the first 10 rounds.
- After round 30 it is almost flat.
- This is on training data only. We have not tested on new data.

Fig. 3. Mean squared error per epoch, as printed by the notebook every 10 epochs. Epoch 99 is not printed.

LIMITS

What it can't do yet

Limit	Fix
No test data, so we don't know how it does on new cases	Train/test split
No bias terms	Add biases
Output can go below 0 or above 1	Sigmoid output
Small model: 3 inputs, 4 neurons	More features, more neurons

> **What was hard for us:** [add 1 or 2 real challenges here]

NEXT STEPS

What we will do next

1 · Test it

- Split into train and test data
- Measure accuracy, precision, recall, F1
- Check the gradients

2 · Improve it

- Add biases
- Sigmoid output with cross-entropy
- More features and neurons

3 · Compare it

- Optimizers, other losses
- Same network in a framework
- Compare the results

The order is our suggestion. The report lists the items but gives no dates.

CONCLUSION

Summary

Done

A small network built by hand. It guesses, checks, and fixes itself. The mistake fell from 0.16 to 0.10.

Still to do

Test on new data, add biases and a sigmoid, then compare with a framework.

Takeaway: the learning loop works. Next we measure it on unseen data.

Sources: Wolberg et al. 1994 and UCI dataset 17 (data); Goodfellow et al. 2016 and Rumelhart et al. 1986 (theory).